



PDF Download
3694715.3695965.pdf
29 March 2026
Total Citations: 0
Total Downloads: 1124



Published: 15 November 2024

Citation in BibTeX format

SOSP '24: ACM SIGOPS 30th Symposium
on Operating Systems Principles
November 4 - 6, 2024
TX, Austin, USA

Conference Sponsors:
SIGOPS

Latest updates: <https://dl.acm.org/doi/10.1145/3694715.3695965>

RESEARCH-ARTICLE

Cookie Monster: Efficient On-Device Budgeting for Differentially-Private Ad-Measurement Systems

PIERRE THOLONIAT, Columbia University, New York, NY, United States

KELLY KOSTOPOULOU, Columbia University, New York, NY, United States

PETER MCNEELY, Columbia University, New York, NY, United States

PRABHPREET SINGH SODHI, Columbia University, New York, NY, United States

ANIRUDH VARANASI, Columbia University, New York, NY, United States

BENJAMIN CASE, Meta, Menlo Park, CA, United States

[View all](#)

Open Access Support provided by:

[Columbia University](#)

[Meta](#)

[The University of British Columbia](#)



Cookie Monster: Efficient On-device Budgeting for Differentially-Private Ad-Measurement Systems

Pierre Tholoniati*
Columbia University

Kelly Kostopoulou*
Columbia University

Peter McNeely
Columbia University

Prabhpreet Singh Sodhi
Columbia University

Anirudh Varanasi
Columbia University

Benjamin Case
Meta Platforms, Inc.

Asaf Cidon
Columbia University

Roxana Geambasu
Columbia University

Mathias Lécuyer
University of British Columbia

Abstract

With the impending removal of third-party cookies from major browsers and the introduction of new privacy-preserving advertising APIs, the research community has a timely opportunity to assist industry in qualitatively improving the Web’s privacy. This paper discusses our efforts, within a W3C community group, to enhance existing privacy-preserving advertising measurement APIs. We analyze designs from Google, Apple, Meta and Mozilla, and augment them with a more rigorous and efficient differential privacy (DP) budgeting component. Our approach, called *Cookie Monster*, enforces well-defined DP guarantees and enables advertisers to conduct more private measurement queries accurately. By framing the privacy guarantee in terms of an individual form of DP, we can make DP budgeting more efficient than in current systems that use a traditional DP definition. We incorporate Cookie Monster into Chrome and evaluate it on microbenchmarks and advertising datasets. Across workloads, Cookie Monster significantly outperforms baselines in enabling more advertising measurements under comparable DP protection.

CCS Concepts: • Security and privacy;

Keywords: Differential Privacy, Budgeting, Measurement

ACM Reference Format:

Pierre Tholoniati*, Kelly Kostopoulou*, Peter McNeely, Prabhpreet Singh Sodhi, Anirudh Varanasi, Benjamin Case, Asaf Cidon, Roxana Geambasu, and Mathias Lécuyer. 2024. Cookie Monster: Efficient On-device Budgeting for Differentially-Private Ad-Measurement Systems. In *ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP ’24)*, November 4–6, 2024, Austin, TX, USA. ACM, New York, NY, USA, 27 pages. <https://doi.org/10.1145/3694715.3695965>



This work is licensed under a Creative Commons Attribution International 4.0 License.
SOSP ’24, November 4–6, 2024, Austin, TX, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1251-7/24/11.
<https://doi.org/10.1145/3694715.3695965>

1 Introduction

Web advertising is undergoing significant changes, presenting a major opportunity to enhance online privacy. For years, numerous entities, often without users’ knowledge, have exploited Web protocol vulnerabilities, such as third-party cookies and remote fingerprinting, to track user activity across the Web. This data has been used to target individuals with ads and assess ad campaign performance. Two key shifts are reshaping this landscape. First, major browsers are making it more difficult to track users across websites. Apple’s Safari and Mozilla’s Firefox blocked third-party cookies in 2019 [20] and 2021 [30], respectively, while Google Chrome will soon facilitate users’ choice of disabling these cookies [6]. Additionally, browsers are strengthening defenses against IP tracking [19] and remote fingerprinting [30, 2, 39].

Second, acknowledging the critical role online advertising plays in the Web economy – and the impossibility of perfect tracking protection – browsers are introducing explicit APIs to measure ad effectiveness and enhance ad delivery while protecting individual privacy. Early designs, like Apple’s PCM [32] and Google’s FLoC [8], focused on intuitive but not rigorous privacy methods, resulting in limited adoption due to poor utility [42] or privacy [17]. Recently, browsers have shifted to theoretically-sound privacy technologies – such as differential privacy (DP), secure multi-party computation (MPC), and trusted execution environments (TEEs) – in the hope of achieving better privacy-utility tradeoffs.

However, substantial challenges remain in implementing these privacy technologies at Web scale. The research community now has a timely opportunity – and responsibility – to assist industry in refining these technologies to deliver both strong privacy protections and meet advertising needs. Only by addressing these challenges can we hope to drive adoption of privacy-preserving APIs, remove incentives for individual tracking, and meaningfully improve Web privacy.

This paper focuses on our efforts to analyze and enhance current *ad-measurement APIs* (a.k.a., attribution-measurement APIs), which enable advertisers to measure and optimize the effectiveness of their ad campaigns based on how often people

*These authors contributed equally to this work.

who view or click certain ads go on to purchase the advertised product. While separate *ad-targeting APIs* are also under development [9], we concentrate on *ad-measurement APIs*.

The W3C’s Private Advertising Technology Community Group (PATCG) [35] is working towards an interoperable standard for private ad-measurement APIs. Leading proposals include Google’s Attribution Reporting API (ARA) [3], Meta and Mozilla’s Interoperable Private Attribution (IPA) [21], Apple’s Private Ad Measurement (PAM) [34], and a hybrid proposal [18]. Our first contribution is a systematization of these proposals into abstract models, followed by a comparative analysis to identify opportunities for improving their privacy-utility tradeoffs (§2).

We focus on the differential privacy (DP) component, present in all four systems. DP is used to ensure advertisers cannot learn too much about any single user through measurement queries. Each system employs a *privacy loss budget*, accounting for the privacy loss incurred by each query. Once the budget is exhausted, further queries are blocked. This process, called *DP budgeting*, is handled centrally in IPA, but in the other systems, DP budgeting is done separately by each device. We observe that this *on-device budgeting* cannot be formalized under standard DP and instead requires a variant, *individual DP* (IDP) or personalized DP [13], for proper formalization. Our formal modeling and analysis of on-device budgeting under IDP form our second contribution (§4).

Through our IDP formalization, we uncover optimizations that enhance utility in on-device budgeting systems, allowing advertisers to execute more accurate queries under the same DP budget. IDP enables devices to maintain their own, separate DP guarantees and to account for privacy loss based on the device’s data. This lets a device deduct zero privacy loss if it lacks relevant data for a query. Notably, one such optimization is already used in ARA, though without formal justification. Our third contribution is providing formal proof for this optimization as well as other, novel optimizations that can further improve the privacy-utility tradeoff.

Our final contribution is a prototype implementation of our optimized DP budgeting system, called *Cookie Monster*, integrated into ARA within Chrome (§3, §5). Cookie Monster is the first ad-measurement system to enforce a fixed, user-time DP guarantee [24], improving on the event-level guarantees of ARA. We evaluate Cookie Monster on microbenchmarks and advertising datasets (§6), showing that it delivers $\times 1.16$ – 2.88 better query accuracy compared to a user-time version of ARA and substantially outperforms IPA, which exhausts its budget very early. Our prototype is available at <https://github.com/columbia/cookiemonster> and has been incorporated into a W3C draft report on privacy-preserving attribution from Mozilla [33].

2 Review of Ad-Measurement APIs

We review the designs of privacy-preserving ad-measurement systems considered for a potential interoperable standard at

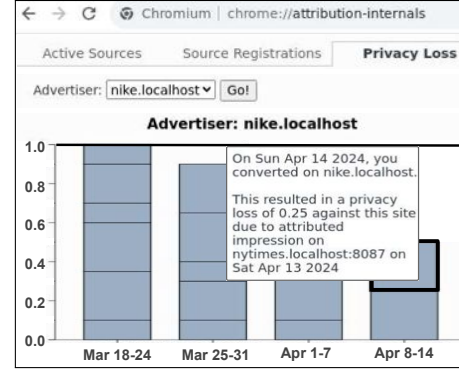


Fig. 1. Privacy loss dashboard. Screenshot from our Chrome implementation of Cookie Monster (minimally edited for visibility).

PATCG: Meta and Mozilla’s IPA, Google’s ARA, Apple’s PAM, and Meta and Mozilla’s Hybrid. ARA and IPA are implemented; PAM and Hybrid exist only as design docs. We abstract their functionality for comparison and articulate the improvement opportunity addressed in this paper.

2.1 Example Scenario

We use a fictitious scenario to illustrate the motivation and requirements of ad-measurement systems from two key perspectives: Ann, a web user, and Nike, an advertiser measuring ad campaign effectiveness. While real-world players like first-party ad platforms (e.g., Meta) and ad-techs (e.g., Criteo) typically run measurement queries on behalf of advertisers, for simplicity, we assume the advertiser performs its own measurements. We discuss the other players in Appendix A.

User perspective. Ann visits various *publisher* sites, such as nytimes.com and facebook.com, where she sees ads. She understands that ads fund the free content she enjoys and occasionally finds them useful, like when she clicked on a Nike ad for running shoes on nytimes.com and later purchased a pair. However, Ann values her privacy and expects *no cross-site tracking*, meaning no site should track her across different websites. She also expects *limited within-site linkability*, preventing even a single site from linking her activities across cookie-clearing browsing sessions (e.g., incognito sessions). Ann accepts that some privacy loss is necessary for effective advertising but expects it to be *explicitly bounded* and *transparently reported* by her browser.

Fig. 1 shows a screenshot of the privacy loss dashboard we developed for Cookie Monster in Chrome, where Ann can monitor the privacy loss resulting from various sites and intermediaries querying her ad interactions, including *impressions* (e.g., ad views and clicks) and *conversions* (e.g., purchases, cart additions). While Ann may not grasp the concept of differential privacy that underpins the reported privacy loss, she trusts her browser to always enforce protective bounds on it.

Advertiser perspective. Nike runs multiple ad campaigns for its running shoes, some emphasizing shock-absorbing technology, others focusing on aesthetics. Nike seeks to understand

which campaigns perform best across different demographics and contexts (e.g., publisher sites, content types). In the past, Nike used third-party cookies and device fingerprinting¹ to track individuals from ad impressions to purchases, attributing purchase value using an *attribution function*, such as last-touch (giving all credit to the last impression) or equal credit (splitting value among recent impressions). Using such *attribution reports* from many users, Nike measured the purchase value attributed to different campaigns and optimized future ad targeting.

Now that third-party cookies are disabled on multiple browsers and fingerprinting is harder, Nike is transitioning to ad-measurement APIs, expecting similar attribution measurements with comparable accuracy. Nike understands that ad measurement has always involved some imprecision (e.g., due to cookie clearing or fraud), so its expectation of accuracy from these APIs is not stringent. Nike plans to conduct numerous attribution measurements over time to adjust to changing user preferences and product offerings. These measurements are single-advertiser summation queries, a key query type that ad-measurement systems aim to support.

2.2 Ad-Measurement Systems

IPA, ARA, PAM, and Hybrid aim to balance user privacy with utility for advertisers and other Web-advertising parties (referred to as *queriers*). Utility is defined as the number of accurate measurement queries a querier can execute under a privacy constraint. Despite variations in terminology, privacy properties, and mechanisms, these systems share key similarities. A commonality is the use of DP techniques, with ARA focusing on event-level DP, while IPA, PAM, and Hybrid emphasize user-time DP. This paper focuses on user-time DP, applied per querier site, as defined in §4.2.3.

Common architecture. The high-level architecture of all four systems is similar (see Fig. 2a). All systems act as intermediaries between user devices and sites. Previously, these parties collected impression and conversion events directly, matched them through third-party cookies, performed attribution, and aggregated reports. To break these privacy-infringing direct data flows, ad-measurement systems interpose a DP querying interface over impression and conversion data.

All systems include three core components: (1) the *attribution function*, which matches conversions to relevant impressions on the same device and assigns conversion value to impressions based on an attribution logic like last-touch; (2) *DP query execution*, which aggregates reports and adds noise for DP guarantees; and (3) *DP budgeting*, which tracks privacy loss from each query using DP composition and enforces a maximum on total privacy loss, called a *DP budget*.

A key difference is where these components are executed. In IPA, all components run off-device within an MPC involving multiple helper servers. In ARA, PAM, and Hybrid,

attribution and DP budgeting occur on-device, while DP query execution is off-device, in an MPC (PAM, Hybrid) or TEE (ARA). The MPC/TEE is trusted not to leak inputs, and the devices are trusted to safeguard their own data. The placement of attribution and DP budgeting is crucial for this paper.

Off-device budgeting (IPA). Fig. 2b illustrates IPA, which operates in a standard centralized-DP setting. The MPC handles all three functions, while the device’s role is limited to generating a *match key* to link impressions and conversions. For example, when nytimes.com sends an ad for Nike shoes to Ann’s device ①, the device responds with a match key, secret-shared and encrypted toward the MPC helper servers ②. When Ann later purchases the shoes on nike.com, her device sends the same key to the MPC, also secret shared and encrypted toward the helpers. Periodically, NYtimes sends batches of encrypted impression match keys to Nike, who cannot directly match these with its conversion match keys due to the encryption and secret sharing. Instead, Nike collects its conversion match keys and NYtimes’ impression match keys into batches and submits them to the MPC, specifying the privacy budget ϵ to spend on the query ③. The MPC checks the budget, matches impressions to conversions, applies the attribution function with an L^1 cap for sensitivity control, aggregates the data, and adds DP noise to enforce ϵ -DP. The MPC tracks and deducts Nike’s privacy budget, refusing further queries once the budget is exhausted until the per-site budget is “refreshed” (e.g., daily).

On-device budgeting (ARA, PAM, Hybrid). Fig. 2c shows the on-device architecture, which operates in a rather non-standard DP setting. While DP query execution occurs centrally on the MPC or TEE, attribution and DP budgeting are done *separately on each device*. Every device maintains a timeseries database of impression and conversion events. When Ann sees an ad for Nike on nytimes.com, her device records it locally ①. Later, when she buys shoes on nike.com, Nike requests an attribution report from her device. Ann’s device checks its database for relevant impressions, applies the attribution function with an L^1 cap, and sends an *attribution report* ②, either secret-shared and encrypted toward the helper parties (for MPC) or directly encrypted to a TEE. Nike aggregates attribution reports from multiple users, submits them to the MPC or TEE, which performs DP aggregation, adding noise based on Nike’s ϵ parameter ③. The MPC/TEE ensures each report is used only once for sensitivity control.

DP budgeting in on-device systems differs from centralized DP by accounting for privacy loss when the advertiser requests a conversion report, prior to query execution. When Nike requests a report, it specifies the ϵ parameter for the future query. The device checks Nike’s budget locally, generates and encrypts the report (with secret sharing if MPC is used), includes ϵ as authenticated data, and deducts ϵ from Nike’s local budget. Since the budget is spent at the device, each report can only be used once, so the device includes a

¹The example is fictitious, as are claims regarding the companies mentioned.

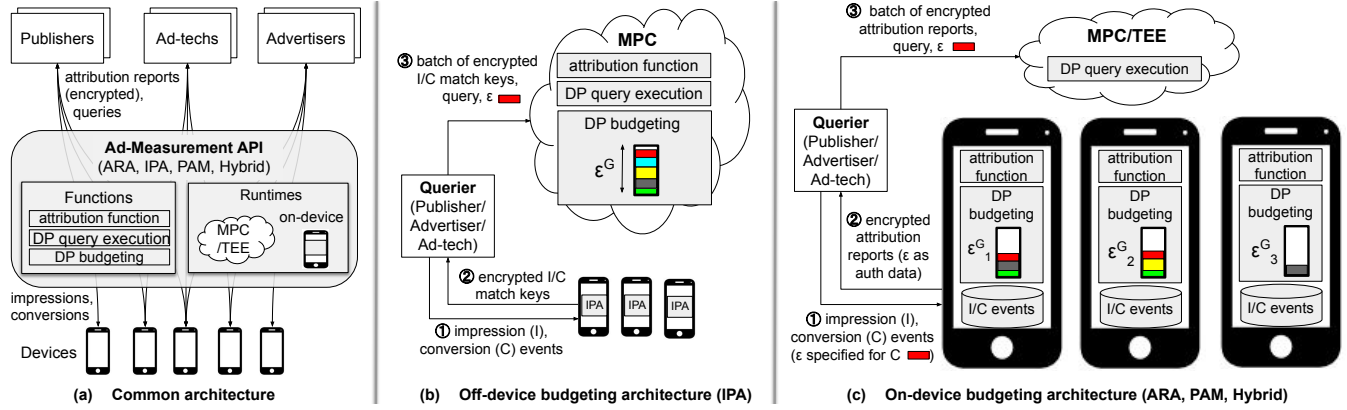


Fig. 2. Architectures of ad-measurement systems. Common structure, with a key difference in where attribution and DP budgeting occur: off-device (IPA) vs. on-device (ARA, PAM, Hybrid).

unique nonce with every report in authenticated data and the MPC/TEE tracks report nonces to prevent reuse.

Threat models. The threat models differ based on whether an MPC or TEE is used. In all cases, MPC/TEE systems are trusted to protect inputs and intermediate states. For MPC, the deployment models assume either a three-party, malicious, honest-majority MPC protocol (IPA, Hybrid) [21] or a two-party malicious protocol (PAM). The querier selects MPC parties from a browser-configured list, typically relatively trusted Web organizations like Cloudflare. The device secret shares the report and encrypts it toward the chosen parties after report generation.

2.3 Improvement Opportunity

On-device budgeting systems offer certain advantages over off-device systems but also present a key challenge, which we aim to address. First, on-device systems can enhance user transparency by putting the user’s device in control of per-site budgets and the tracking of privacy losses incurred by the user due to specific attribution reports the device releases to various querier sites, as seen in the Cookie Monster privacy loss dashboard (Fig. 1). In contrast, in IPA, the device can only track the encrypted match keys returned by the device, not the specific privacy losses users incur through subsequent matching and aggregation in the MPC.

Second, on-device systems allow for finer-grained budgeting. While off-device systems enforce a global site-wide budget ϵ^G , on-device systems maintain a per-device budget ϵ_d^G , which is only consumed for queries involving that device. This granularity enables Nike, for instance, to continue querying other users’ reports even if it exhausts Ann’s budget. However, this behavior requires formalization under the less standard (but equally protective) privacy definition known as individual DP (IDP) [13], which allows enforcement of a separate privacy guarantee for each device.

The challenge lies in formalizing the data, query, and system model that capture the behavior of on-device ad-measurement systems, and in proving its IDP properties. This formalization then opens opportunities for further optimizing

DP budgeting in on-device systems by deducting privacy loss based on the device’s data. However, it also requires keeping the remaining privacy budgets on each device private, as revealing these budgets leaks data. This paper presents a formally-justified, practical and efficient DP budgeting module, *Cookie Monster*, designed for on-device systems like ARA, PAM, and Hybrid, which maximizes utility while maintaining DP guarantees.

3 Cookie Monster Overview

The design of Cookie Monster is guided by three principles. First, it must enforce well-defined DP guarantees at an industry-accepted granularity. We adopt a fixed “user-time” DP guarantee for each querier, supported by IPA, PAM, and Hybrid, and recognized by Apple, Meta, and Mozilla as the minimum acceptable. Second, Cookie Monster must support similar use cases and queries as existing systems. We focus on the single-advertiser measurement query from §2.1, though we briefly discuss in Appendix A how a multi-advertiser optimization query might apply. Finally, Cookie Monster must not introduce new vectors for illicit tracking, given increasing browser efforts to prevent tracking both across sites and within-site across cookie refreshes.

Fig. 3 presents Cookie Monster’s architecture with an example execution overlaid. We describe each aspect below.

3.1 Architecture

Cookie Monster adopts on-device budgeting, similar to ARA, PAM, and Hybrid. DP query execution occurs off device, in an MPC or TEE, trusted not to leak inputs or intermediate states. Since Cookie Monster does not modify this component, it is omitted from Fig. 3; we think of it as a trusted *aggregation service*. Cookie Monster modifies the on-device component, based on ARA in our prototype. While the external APIs remain unchanged, we modify: (1) the on-device events database to support a “user-time” guarantee, and (2) the internals of the attribution function and DP budgeting to enforce this guarantee efficiently.

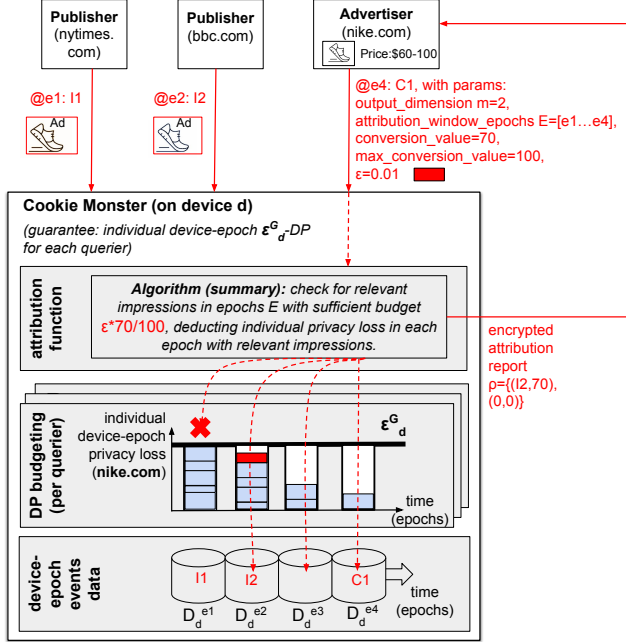


Fig. 3. Cookie Monster architecture and example execution (red overlay). §3.1 describes the architecture and §3.2 the example execution. Notation: $@e_1 : I_1$ indicates that Ann’s device receives an impression I_1 of a Nike shoe ad from nytimes.com in epoch e_1 . Red dotted arrows show the attribution function’s search for impressions over epochs $e_1 - e_4$.

Cookie Monster enforces *individual device-epoch* ϵ_d^G -DP for each querier site, formally defined in §4.2.3. This device-epoch granularity aligns with traditional “user-time” from DP literature [24, 26, 27], though we rename it to reflect that a user’s complete activity is not directly observable by a device or browser, the scope in which Cookie Monster operates. We partition the on-device events database into time-based *epochs*, such as weeks or months. In each epoch e , device d collects impression and conversion events into a *device-epoch database* D_d^e . Queriers submit multiple queries over time, accessing data from one or more epochs. For each epoch e , Cookie Monster ensures that no querier learns more about device d ’s data in e than permitted by an ϵ_d^G -DP guarantee.

The *DP budgeting* in Cookie Monster is implemented using privacy filters [37], which ensure that the cumulative privacy loss from a series of queries does not exceed a pre-specified budget. For each querier, Cookie Monster maintains multiple filters – one for each device-epoch database. Fig. 3 shows these filters for nike.com. Each filter is initialized with a privacy budget ϵ_d^G and monitors cumulative privacy loss for queries involving data from that epoch.

In on-device systems, privacy loss is accounted for when the attribution report is generated, not when the query is executed. The *attribution function* is responsible for generating these reports. Upon a conversion, the function checks for relevant impressions in the device-epoch databases within a specified attribution window. Privacy filters prevent use of impression data from epochs with insufficient budget.

For epochs with sufficient budget, the filter allows access to the device-epoch data and deducts privacy loss. Under standard centralized DP, this loss would be ϵ , the DP parameter enforced later by the MPC or TEE during aggregation. However, our theoretical analysis of on-device budgeting reveals that viewing the system under an individual-DP lens opens opportunities to optimize privacy accounting, often allowing deductions of “less than ϵ .” §4 outlines our theoretical analysis, a major contribution in this paper. We dedicate the remainder of this section to providing the systems view of our theory, including an execution example (§3.2), Cookie Monster’s algorithm, which is backed by our theory (§3.3), and a discussion on mitigating IDP-induced bias (§3.4).

3.2 Execution Example

The red overlay in Fig. 3 illustrates the attribution function’s operation for the example from §2.1. Ann receives two impressions of Nike shoe ads: one in epoch e_1 and another in e_2 , with no impressions in e_3 . Later, in epoch e_4 , Ann buys the shoes, and nike.com registers a conversion $C1$. It requests an attribution report with parameters: the set of epochs E to search for impressions, the maximum number of impressions m to attribute value to, the conversion value ($\$70$), and ϵ , the privacy parameter enforced by the MPC or TEE when executing the aggregation query.

The shoes’ price ranges by color, with a maximum of $\$100$. While Ann’s conversion is $\$70$, Nike’s query will include conversions up to $\$100$. Thus, for a summation query with the Laplace mechanism, the noise added to the aggregate depends on $100/\epsilon$, where 100 is the *global sensitivity* of the summation (i.e., the largest change *any* device-epoch can contribute). Ann, with a purchase of $\$70$, can only contribute up to $\$70$ across her device-epochs.

Here, IDP lets us optimize privacy loss based on *individual sensitivity*, the maximum change that a *specific* device-epoch can make on the query output. In this case, Ann’s device only deducts $\epsilon' = \$70/\$100 * \epsilon$ from the privacy filters of the epochs in the attribution window E . This is one optimization enabled by IDP. Another is that if no relevant impressions exist in an epoch (e.g., e_3 in Fig. 3), we need not deduct anything, since the individual sensitivity for that epoch is 0 and thus its privacy loss is also 0. §4.3 formalizes global and individual sensitivities and details further optimizations.

In Fig. 3, Cookie Monster’s attribution function checks epochs $e_1 - e_4$ for relevant impressions. In e_1 , access to data $D_d^{e_1}$ is denied because the filter has exhausted nike.com’s budget. In e_2 , the filter allows access, and a relevant impression I_2 is found, deducting ϵ' (shown as a red square in the e_2 filter). In e_3 , there is budget, but no relevant impression is found, so no deduction occurs. Finally, in e_4 , where the conversion happened but no impression occurred, then through a formalization of publicly available information that we support (§4.1), we can justify that no privacy loss occurs in e_4 .

The final attribution report assigns the \$70 value to the single impression I_2 and includes a null value for the second attribution, as Nike requested two. If no impressions were found, or Nike also ran out of budget in e_2 , the attribution function would return a report with two null values to avoid leaking information about ad presence.

3.3 Algorithm

Listing 1 shows how Cookie Monster computes an attribution report. The `compute_attribution_report` function receives an `attribution_request`, which encapsulates all querier-provided parameters, sanitized by the device. Key parameters include:

1. the window of epochs to search for relevant events (epochs parameter);
2. the requested privacy budget (requested_epsilon);
3. logic for selecting relevant events (select_relevant_events);
4. the attribution policy, such as last-touch or equal-credit (compute_attribution);
5. two global sensitivity parameters: `report_global_sensitivity`, the maximum change a device-epoch can make to the output of the report generation function, and `query_global_sensitivity`, the maximum across all devices and reports;
6. p-norm, based on the DP mechanism in MPC/TEE, e.g., 1-norm for Laplace and 2-norm for Gaussian.

All parameters follow a predefined protocol, and while the algorithm is general enough to handle different mechanisms and p-norm sensitivities, our DP result (Thm. 1) focuses on pure DP, assuming the Laplace mechanism and L_1 sensitivity. Computing an attribution report consists of four steps.

Step 1: Cookie Monster invokes the querier-provided `select_relevant_events` to select relevant events from each separate epoch in the attribution window, such as impressions with a specific campaign ID.

Step 2: For each epoch, Cookie Monster computes the individual privacy loss resulting from the querier’s query, following the IDP optimizations in Thm. 4. Three cases:

1. if the epoch has no relevant events, privacy loss is zero;
2. if a single epoch is considered, privacy loss is proportional to the L_p -norm of the attribution function output;
3. if multiple epochs are considered, privacy loss is proportional to the report’s global sensitivity.

The privacy loss is scaled by `requested_epsilon` and the query’s global sensitivity. In §3.2, the report’s global sensitivity is 70, and the query’s global sensitivity is 100.

Step 3: For each epoch, we attempt to deduct the computed privacy loss from the querier’s budget for that epoch, ensuring atomic, thread-safe checks. If the filter has sufficient budget, the epoch’s events are used for attribution; otherwise, they are dropped. The justification for dropping contributions is provided in Theorem 1.

Step 4: The attribution function is applied across events from all epochs, following the querier’s policy. The device ensures that the attribution computation: (1) respects the querier’s specified `report_global_sensitivity` by clipping the attribution histogram to ensure its L_p -norm is $\leq$ `report_global_sensitivity`, and (2) produces encrypted outputs indistinguishable from others. For (2), the device ensures a fixed dimension for the attribution report by padding or dropping elements. For instance, if only one relevant impression is found but two are requested, the output vector is padded with a null entry.

```
# Global variables: events_database, privacy_filters.
def compute_attribution_report(attribution_request):
    relevant_events_per_epoch = {}
    for epoch in attribution_request.epochs:
        relevant_events = attribution_request.select_relevant_events(
            events_database[epoch]) # Step 1
        individual_privacy_loss = compute_individual_privacy_loss(
            relevant_events, attribution_request) # Step 2
        filter_status = privacy_filters[attribution_request.
            querier_site][epoch].check_and_consume(
                individual_privacy_loss) # Step 3
        if filter_status == "out_of_budget":
            relevant_events = {}
            relevant_events_per_epoch[epoch] = relevant_events
        return attribution_request.compute_attribution(
            relevant_events_per_epoch) # Step 4

def compute_individual_privacy_loss(epoch_events,
    attribution_request):
    if epoch_events == {}: # Case 1 in Theorem 4
        return 0
    if len(attribution_request.epochs) == 1: # Case 2 in Theorem 4
        individual_sensitivity = attribution_request.pnorm(
            attribution_request.compute_attribution(relevant_events))
    else: # Case 3 in Theorem 4
        individual_sensitivity = attribution_request.
            report_global_sensitivity
    return attribution_request.requested_epsilon *
        individual_sensitivity / attribution_request.
            query_global_sensitivity
```

Code Listing 1. Cookie Monster Algorithm

For the example in §3.2, this algorithm is invoked with an `attribution_request` where `querier_site` = “nike.com,” `epochs` = $[e_1 - e_4]$, `report_global_sensitivity` = 70, `query_global_sensitivity` = 100. Function `select_relevant_events` filters impressions by campaign ID, `pnorm` returns the L_1 -norm of the attribution histogram, and `compute_attribution` divides the conversion value of 70 across at most two impressions, padding with nulls as needed. This attribution function has sensitivity 70.

3.4 Bias Implications of IDP

The execution example and algorithm demonstrate Cookie Monster’s budget savings, confirmed in Section 6, where we show that these savings allow more accurate queries than ARA and IPA under the same privacy guarantees. However, IDP can introduce bias into query results. Since privacy loss and remaining budgets depend on data, they must remain hidden from advertisers. When a device exhausts its budget for an epoch, it continues participating in queries with “null” data, protecting privacy but potentially introducing bias. For example, Nike’s report should have included two impressions, but running out of budget in epoch e_1 meant I_1 wasn’t returned, altering the report undetectably.

This bias is a general challenge for all systems operating on IDP, including all existing ad-measurement systems with on-device budgeting – although this challenge is not always acknowledged or handled. Indeed, ARA incorporates code to send null reports when budgets are exhausted and its documentation states that these nulls must be sent to preserve privacy [16]. Such nulls would add bias to query results. In absence of proper IDP formulation, a rudimentary justification we have seen for sending nulls in on-device systems is to prevent revealing budget exhaustion, which could facilitate remote fingerprinting, a concern actively addressed by browsers. Our paper reveals a deeper issue: these systems inherently operate under IDP, and IDP systems must keep budgets hidden, which can lead to bias. Acknowledging this bias opens pathways to mitigate it.

Any (DP or IDP) system must tolerate some error. In ad measurement, high error tolerance is common due to factors like tracking inaccuracies and fraud. The goal is to equip queriers with tools that rigorously bound errors from both DP noise and IDP bias, allowing for informed decision-making. Previous work on centralized-budgeting IDP has developed methods to bound bias using global sensitivity [14] and periodic DP counting queries [45, 14]. These approaches require adaptation to on-device budgeting, given the lack of centralized privacy-loss tracking and non-i.i.d. report sampling. We leave it for future work to develop advanced bias-management tools and here only present a rudimentary approach, which we implement in Cookie Monster and evaluate in §6.5 as a proof-of-concept that bias can be effectively managed in on-device budgeting systems.

Our approach adds a *side query* to each attribution query, which bounds potential error from out-of-budget epochs. With each report, the querier requests a boolean flag indicating whether the report could be affected by an out-of-budget epoch. This flag is bundled with the attribution report, secret-shared, and encrypted toward the MPC/TEE. The querier receives a DP-aggregated count of how many reports could be erroneous out of its total batch. With the count, the querier computes a high-probability upper bound on the error from both DP noise and IDP bias. The querier can then filter the results of its queries based on this error bound, ignoring those with unacceptable error. Formalization and proof of this mechanism’s correctness are deferred to Appendix F.

Consider last-touch attribution. If no epoch in the attribution window is out of budget or an impression is found in a later epoch, the device returns a 0-valued error assessment, indicating no bias. If no impression is found in epochs later than the out-of-budget epoch, the device returns a 1-valued error assessment, signaling potential bias. This information is encrypted and only accessible to the querier after DP aggregation by the MPC/TEE.

This mechanism lets queriers manage IDP-induced error rigorously, though it consumes additional privacy budget. In

Steps 3 and 4 of Listing 1, each epoch that is not out of budget must deduct privacy loss for the side query. Fortunately, since the side query is a count query with lower sensitivity than the main query, Cookie Monster’s optimizations still provide benefits. Our evaluation shows that even with bias detection, Cookie Monster consumes less privacy and incurs lower errors compared to ARA and IPA (§6.5).

4 Formal Modeling and Analysis

This section outlines the theoretical analysis behind Cookie Monster’s design, divided into three parts: §4.1 introduces a formal model that captures the behavior of on-device budgeting systems, including Cookie Monster but also ARA and PAM. §4.2 analyzes this model under IDP, proving that Cookie Monster bounds cross-site leakage and within-site linkability. Finally, §4.3 details and justifies the optimizations enabled by IDP, both ones inherently employed in ARA and new ones that our theory uncovers.

4.1 Formal System Model

To rigorously analyze privacy properties and identify optimization opportunities in on-device budgeting systems for ad measurement, we must establish a formal model of their behavior. Current ad-measurement systems lack such models, preventing formal analysis or justification of optimizations. Although our model is tailored to Cookie Monster, it can also serve as a foundation for analyzing other systems.

We define the data and queries Cookie Monster operates on, from the perspective of a fixed querier (e.g., advertiser, publisher, or ad-tech). Appendix §C formalizes the end-to-end algorithm, incorporating these models and the Cookie Monster behavior outlined in §3. Since this algorithm is used solely to prove the DP guarantees in §4.2, we omit it here.

4.1.1 Data Model

Our data model is based on conversion and impression events collected by user devices and grouped by the time epoch in which they occurred. We view the data available to queriers as a database of such device-epoch groups of events, coming from many devices and defined formally as follows.

Conversion and impression events (F). Consider a domain of impression events I and a domain of conversion events C . A set of impression and conversion events F is a subset of $I \cup C$. The powerset of events is $\mathcal{P}(I \cup C) := \{F : F \subset I \cup C\}$.

Device-epoch record (x). Consider a set of epochs $\mathcal{E}$ and a set of devices $\mathcal{D}$. We define the domain for device-epoch records $\mathcal{X} := \mathcal{D} \times \mathcal{E} \times \mathcal{P}(I \cup C)$. That is, a *device-epoch record* $x = (d, e, F)$ contains a device identifier d , an epoch identifier e , and a set of impression and conversion events F .

Database (D). A *database* is a set of device-epoch records, $D \subset \mathcal{X}$, where a device-epoch appears at most once. That is, $\forall d, e \in \mathcal{D} \times \mathcal{E}, |\{F \subset I \cup C : (d, e, F) \in D\}| \leq 1$. We denote the set of all possible databases by $\mathbb{D}$. This will be the domain of queries in Cookie Monster. Given a database $D \in \mathbb{D}$ and

$x \in X$, $D + x$ denotes that device-epoch record x is added to database D that initially did not include it.

Device-epoch events data (D_d^e, D_d^E). Given a database $D \in \mathbb{D}$, we define $D_d^e \subset I \cup C$ as $D_d^e = F$ if there exist (a unique) F such that $(d, e, F) \in D$, and $D_d^e = \emptyset$ otherwise. Think of this as the event data of device d at epoch e . We also define $D_d^E := (D_d^e)_{e \in E} \in \mathcal{P}(I \cup C)^{|E|}$ the events of device d over a set of epochs E (typically a contiguous window of epochs).

Public events (P). A key innovation in Cookie Monster’s data model is to support incorporation of side information that can be reliably assumed as available to the querier. For example, an advertiser such as Nike can reliably know when someone places a product into a cart (i.e., a conversion occurred), though depending on whether the user is logged in or not, Nike may or may not know who did that conversion.

We model such side information as a domain of *public events* for a querier, denoted $P \subseteq I \cup C$. P is a subset of all possible events, that will be disclosed to the querier if they occur in the system. We do *not* assume that the querier knows the devices on which events in P occur, and different queriers can have knowledge about different subsets of events. Such side information is typically not modeled explicitly in DP systems, as DP is robust to side information. Cookie Monster also offers such robustness to generic side information. However, we find that additionally modeling the “public” events known to the querier has two key benefits. First, it opens DP optimizations that leverage this known information to consume less privacy budget. Second, it lets us formally define within-site linkability and adapt our design to provide a DP guarantee against such linkability.

4.1.2 Query Model

In on-device systems, queries follow a specific format: first the attribution function runs locally to generate an attribution report, on a set of devices with certain conversions; then, the MPC sums the reports together and returns the result with DP noise. Formally, we define three concepts: attribution function, attribution report, and query.

Attribution function, a.k.a. attribution (A). Fix a set of events relevant to the query $F_A \in \mathcal{P}(I \cup C)$, and $k, m \in \mathbb{N}^*$ where k is a number of epochs. An *attribution function* is a function $A : \mathcal{P}(I \cup C)^k \rightarrow \mathbb{R}^m$ that takes k event sets $F_1, \dots, F_k$ from k epochs and outputs an m -dimensional vector $A(F_1, \dots, F_k)$, such that only *relevant events* contribute to A . That is, for all $(F_1, \dots, F_k) \in \mathcal{P}(I \cup C)^k$, we have:

$$A(F_1, \dots, F_k) = A(F_1 \cap F_A, \dots, F_k \cap F_A).$$

Attribution report, a.k.a. report (ρ). This is where the non-standard behavior of on-device budgeting systems, which deduct budget only for devices with specific conversions, becomes apparent. Intuitively, we might consider attribution reports as the “outputs” of an attribution function. However, in the formal privacy analysis, we must account for the fact that only certain devices self-select to run the attribution function (and thus deduct budget). We model this in two steps. First, we

introduce a conceptual *report identifier*, r , a unique random number that the device producing this report generates and shares with the querier at report time.

Second, we define an *attribution report* as a function over the whole database D , that returns the result of an attribution function A for a set of epochs E *only for one specific device d as uniquely identified by a report identifier r* . Formally, $\rho_r : D \in \mathbb{D} \mapsto A(D_d^E)$. At query time, the querier selects the report identifiers it wants to include in the query (such as those associated with a type of conversion the querier wants to measure), and devices *self-select* whether to deduct budget based on whether they recognize themselves as the generator of any selected report identifiers. Defining attribution reports on D lets us account for this self-selection in the analysis.

Query (Q). Consider a set of report identifiers $R \subset \mathbb{Z}$, and a set of attribution reports $(\rho_r)_{r \in R}$ each with output in $\mathbb{R}^m$. The *query* for $(\rho_r)_{r \in R}$ is the function $Q : \mathbb{D} \rightarrow \mathbb{R}^m$ is defined as $Q(D) := \sum_{r \in R} \rho_r(D)$ for $D \in \mathbb{D}$.

4.1.3 Instantiation in Example Scenario

To make our data and query models concrete, we instantiate the scenarios from §2.1.

User Ann’s data, together with that of other users, populates dataset D . Each device Ann owns has an identifier d , and events logged from epoch e go into observation $x = (d, e, F)$. $F = I \cup C$ is the set of all events logged on that device during that epoch, including impressions (I) shown to Ann by various publishers, and conversions (C) with various advertisers. Other devices of Ann, other epochs, and other users’ device-epochs, constitute other records in the database.

The advertiser, Nike, can observe some of Ann’s behavior on its site. As a result, any such behavior logged in C on nike.com constitutes public information for querier Nike. This might include purchases, putting an item in the basket, as well as associated user demographics (e.g., when Ann is logged-in). However, Nike cannot observe impression or conversion events on other websites. As a result, for this querier $P = C_{\text{Nike}}$, which denotes all possible events that can be logged on nike.com. Each actual event in this set (e.g., $F \cap C_{\text{Nike}}$, including Ann’s purchase) is associated with an identifier r in Cookie Monster. Using these identifiers, Nike can analyze the relative effectiveness of two ad campaigns a_1 and a_2 on a given demographics for a product p , such as the shoes Ann bought. First, Nike defines the set of relevant events for the shoe-buying conversion; these are any impressions of a_1 and a_2 . Nike uses these relevant events in an attribution function $A : \mathcal{P}(I \cup C)^{|E|} \rightarrow \mathbb{R}^2$ that looks at epochs in E and returns, for example, the count (or value) of impression events corresponding to ads a_1 and a_2 . Third, using the set of report identifiers r from purchases of p from users in the target demographic, Nike constructs a query Q that will let it directly compare the proportion of purchases associated with ad campaign a_1 versus campaign a_2 .

4.2 IDP Formulation and Guarantees

With Cookie Monster’s data and query models defined, we now formalize and prove its privacy guarantees using individual DP. After introducing our neighboring relation in §4.2.1, we briefly define traditional DP for reference in §4.2.2, followed by individual DP in §4.2.3. In §4.2.4, we state the IDP guarantees for Cookie Monster, which imply protection against both cross-site tracking and within-site linkability.

4.2.1 Neighboring Databases

A DP guarantee establishes the neighboring database relation, determining the unit of protection. In our case, this unit is the device-epoch record. To account for the existence of public event data (§4.1.1), we constrain neighboring databases to differ by one device-epoch record *while preserving public information*. This ensures that a database containing an arbitrary device-epoch record is indistinguishable from a database containing a device-epoch record with the same public information but no additional data.

Neighboring databases under public information ($D \sim_x^P D'$). Given $D, D' \in \mathbb{D}$, $x = (e, d, F) \in \mathcal{X}$ and $P \subset \mathcal{I} \cup \mathcal{C}$, we write $D \sim_x^P D'$ if there exists $D_0 \in \mathbb{D}$ such that $\{D, D'\} = \{D_0 + (e, d, F), D_0 + (e, d, F \cap P)\}$. This definition corresponds to a replace-with-default definition [14] combined with Label DP [15]. Although public data is baked into our neighboring relation, which makes it specific to each individual querier, we have proven that composition across queriers is still possible, which is important to reason about collusion (Appendix §D.3).

4.2.2 DP Formulation (for Reference)

In DP, noise must be applied to query results based on the query’s *sensitivity*—the worst-case difference between two neighboring databases. Traditional DP mechanisms rely on global sensitivity.

Global sensitivity. Fix a query $q : \mathbb{D} \rightarrow \mathbb{R}^m$ for some m (so q could be either a query or an individual report in our formulation). We define the *global L_1 sensitivity* of q as follows:

$$\Delta(q) := \max_{D, D' \in \mathbb{D} : \exists x \in \mathcal{X}, D' = D+x} \|q(D) - q(D')\|_1. \quad (1)$$

Device-epoch DP. When scaling DP noise to the global sensitivity under our neighboring definition, we can provide device-epoch DP. Fix $\epsilon > 0$ and $P \subset \mathcal{I} \cup \mathcal{C}$. A randomized computation $\mathcal{M} : \mathbb{D} \rightarrow \mathbb{R}^m$ satisfies *device-epoch ϵ -DP* if for all databases $D, D' \in \mathbb{D}$ such that $D \sim_x^P D'$ for some $x \in \mathcal{X}$, for any set of outputs $S \subseteq \mathbb{R}^m$ we have $\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S]$. This is the traditional DP definition, instantiated for our neighboring relation.

4.2.3 IDP Formulation

Since queries are aggregated from reports computed on-device with known data, we would prefer to scale the DP noise to the individual sensitivity, which is the worst case change in a query result triggered by the specific data for which we are computing a report.

Individual sensitivity. Fix a function $q : \mathbb{D} \rightarrow \mathbb{R}^m$ for some m (so q could be either a query or an individual report in our formulation) and $P \subset \mathcal{I} \cup \mathcal{C}$. Fix $x \in \mathcal{X}$. We define the *individual L^1 sensitivity* of q for x as follows:

$$\Delta_x(q) := \max_{D, D' \in \mathbb{D} : D' = D+x} \|q(D) - q(D')\|_1. \quad (2)$$

While we cannot directly scale the noise to individual sensitivity, we can scale the on-device budget consumption using this notion of sensitivity. That is, for a fixed and known amount of noise that will be added to the query, a lower individual sensitivity means that less budget is consumed from a device-epoch. This approach provides a guarantee of individual DP [13, 14] for a device-epoch, defined as follows.

Individual device-epoch DP. Fix $\epsilon > 0$, $P \subset \mathcal{I} \cup \mathcal{C}$, and $x \in \mathcal{X}$. A randomized computation $\mathcal{M} : \mathbb{D} \rightarrow \mathbb{R}^m$ satisfies *individual device-epoch ϵ -DP* for x if for all databases $D, D' \in \mathbb{D}$ such that $D \sim_x^P D'$, for any set of outputs $S \subseteq \mathbb{R}^m$ we have $\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S]$.

Intuitively, IDP ensures that, from the point of view of a fixed device-epoch x , the associated data F is as hard to recover from query results as it would be under DP.

4.2.4 IDP Guarantees

Through IDP, we prove two main properties of Cookie Monster: (1) **Individual DP guarantee**, which implies bounds on *cross-site leakage*, demonstrating that the API cannot be used to reveal cross-site activity; and (2) **Unlinkability guarantee**, which implies bounds on *within-site linkability*, demonstrating that the API cannot be used even by a first-party site to distinguish whether a set of events is all on one device vs. spread across two devices. Proofs are in Appendix §D.

For the IDP guarantee, we give two versions. First, a stronger version under a mild constraint on the class of allowed queries, specifically that $\forall i, \forall F, A(F_1, \dots, F_{i-1}, F_i \cap P, F_{i+1}, \dots, F_k) = A(F_1, \dots, F_{i-1}, \emptyset, F_i, \dots, F_k)$. A sufficient condition is to ensure that queries leverage public events only through their report identifier, *i.e.*, $F_A \cap P = \emptyset$. The queries from the scenarios we consider (§2.1) satisfy this property. Second, a slightly weaker version of the DP guarantee with increased privacy loss, but with no constraints on the query class, which is useful when considering colluding queriers.

Theorem 1 (Individual DP guarantee). *Fix a set of public events $P \subset \mathcal{I} \cup \mathcal{C}$, and budget capacities $(\epsilon_d^G)_{d \in \mathcal{D}}$. **Case 1:** If all the queries use attribution functions A satisfying $\forall i, \forall F, A(F_1, \dots, F_{i-1}, F_i \cap P, F_{i+1}, \dots, F_k) = A(F_1, \dots, F_{i-1}, \emptyset, F_i, \dots, F_k)$, then for $x \in \mathcal{X}$ on device d , Cookie Monster satisfies individual device-epoch ϵ_d^G -DP for x under public information P . **Case 2:** For general attribution functions, Cookie Monster satisfies individual device-epoch $2\epsilon_d^G$ -DP for x under public information P .*

Intuitively, the information gained on cross-site (private to the querier) events in device-epoch x under the querier’s queries is bounded by ϵ_x^G (or $2\epsilon_x^G$ without query constraints).

Theorem 2 (Unlinkability guarantee). Fix budget capacities $(\epsilon_d^G)_{d \in \mathcal{D}}$. Take any $d_0, d_1 \in \mathcal{D}$, $e \in \mathcal{E}$, and $F_1 \subset F_0$. Denote $x_0 := (d_0, e, F_0)$, $x_1 := (d_1, e, F_1)$, $x_2 := (d_0, e, F_0 \setminus F_1) \in \mathcal{X}$. Take any $D \in \mathbb{D}$ such that $(d_0, e) \notin D$ and $(d_1, e) \notin D$, and any instantiation $\mathcal{M}$ of Cookie Monster. For all $S \subset \text{Range}(\mathcal{A})$ we have: $\Pr[\mathcal{M}(D+x_0) \in S] \leq e^{2\epsilon_{d_0}^G + \epsilon_{d_1}^G} \Pr[\mathcal{M}(D+x_1+x_2) \in S]$.

Intuitively, linking a set of events across two devices—compared to detecting these events on one device—is only made easier by the amount of budget on the second device; Cookie Monster does not introduce additional privacy loss for linkability, above what is revealed through DP queries.

4.3 IDP Optimizations

IDP allows discounting the DP budget based on individual sensitivity, which is never greater but often smaller than global sensitivity. The easiest way to grasp this opportunity is to visualize and compare the definitions of global and individual sensitivities for reports and queries. Recall that Cookie Monster enforces a bound on reports by capping each coordinate in the attribution function’s output to a querier-provided maximum. Given this cap, we prove the following formulas for both sensitivities (proofs in Appendix §E):

Theorem 3 (Global sensitivity of reports and queries).

Fix a report identifier r , a device d_r , a set of epochs E_r , an attribution function A and the corresponding report $\rho : D \mapsto A(D_{d_r}^{E_r})$. We have:

$$\Delta(\rho) = \max_{i \in [k], F_1, \dots, F_k \in \mathcal{P}(\mathcal{I} \cup \mathcal{C})} \|A(F_1, \dots, F_k) - A(F_1, \dots, F_{i-1}, \emptyset, F_{i+1}, \dots, F_k)\|_1$$

Next, fix a query Q with reports $(\rho_r)_{r \in R}$ such that each device-epoch participates in at most one report. We have $\Delta(Q) = \max_{r \in R} \Delta(\rho_r)$.

Theorem 4 (Individual sensitivity of reports and queries).

Fix a device-epoch record $x = (d, e, F) \in \mathcal{X}$. Fix a report identifier r , a device d_r , a set of epochs $E_r = \{e_1, \dots, e_k\}$, an attribution function A with relevant events F_A , and the corresponding report $\rho : D \mapsto A(D_{d_r}^{E_r})$.

We have: $\Delta_x(\rho) = \max_{F_1, \dots, F_{i-1}, F_{i+1}, \dots, F_k \in \mathcal{P}(\mathcal{I} \cup \mathcal{C})} \|A(F_1, \dots, F_{i-1}, F, F_{i+1}, \dots, F_k) - A(F_1, \dots, F_{i-1}, \emptyset, F_{i+1}, \dots, F_k)\|_1$ if $d = d_r$ and $e = e_i \in E_r$, and $\Delta_x(\rho) = 0$ otherwise.

In particular,

$$\Delta_x(\rho) \leq \begin{cases} 0 & \text{if } d = d_r, e \in E_r \text{ and } F \cap F_A = \emptyset \\ \|A(F) - A(\emptyset)\|_1 & \text{if } d = d_r \text{ and } E_r = \{e\} \\ \Delta(\rho) & \text{if } d = d_r, e \in E_r \text{ and } F \cap F_A \neq \emptyset \end{cases}$$

Next, fix a query Q with reports $(\rho_r)_{r \in R}$. Then we have: $\Delta_x(Q) \leq \sum_{r \in R} \Delta_x(\rho_r)$. In particular, if x participates in at most one report ρ_r , then: $\Delta_x(Q) = \Delta_x(\rho_r)$.

This theorem justifies both the inherent optimization used by all on-device systems and the new optimizations added in Cookie Monster.

Inherent on-device optimization. The condition $d = d_r$ in Thm. 4 explains why, under IDP, on-device budgeting systems

deduct privacy loss only for devices that participate in a query. This is more efficient than off-device systems like IPA, which, under traditional DP, must deduct budget based on $\Delta(Q)$ from all devices, regardless of their participation (Thm. 3).

New optimization examples. First, devices that participate in a query but have no relevant data (i.e., $F \cap F_A = \emptyset$ or $A(F) = A(\emptyset)$ in Thm. 4) do not incur budget loss. This is why, in the example from § 3.2, we don’t deduct from epoch e_3 , which has no Nike impressions. Second, a device’s individual sensitivity depends only on reports it participates in ($\Delta_x(Q) = \Delta_x(\rho_r)$), whereas global sensitivity depends on all reports in the query ($\Delta(Q) = \max_{r \in R} \Delta(\rho_r)$). For instance, since the report ρ typically depends on the public information $F \cap P$ of a record (d, e, F) , we use a \$70 cap instead of \$100 in the Nike example. Third, if an attribution spans only one epoch (or is broken into single-epoch reports), individual sensitivity can be further reduced based on the private information F . For example, if Nike measures the average impression-to-conversion delay (0 to 7 days) in a single epoch and a record x has one impression only 1 day before the conversion, its individual budget will be 1/7th of the global budget.

5 Chrome Prototype

We integrated Cookie Monster into Google Chrome by modifying ARA. We disabled ARA’s impression-level budgeting, added epoch support, and extended ARA’s database to include a table for privacy filters for each epoch-querier pair. Unlike ARA, which supports only last-touch attribution and fetches only the latest impression, our implementation retrieves all impressions related to the conversion, groups them by epoch, and identifies epochs with no relevant data to avoid unnecessary budget consumption.

6 Evaluation

We seek to answer three key questions:

- Q1:** How do optimizations impact budget consumption?
- Q2:** How do optimizations impact query accuracy?
- Q3:** How effective is bias measurement?

We also evaluate Cookie Monster’s runtime overhead, but provide these results in Appendix B as they are not vital to our main hypotheses in this paper.

6.1 Methodology

We evaluate Cookie Monster on three datasets—a microbenchmark and two realistic advertising datasets from PATCG and Criteo—and compare its privacy budget consumption and query accuracy against two baselines. The first baseline is **IPA-like**, our own prototype implementing IPA’s centralized budgeting and query execution. The second is **ARA-like**, a version of ARA providing device-epoch-level guarantees. ARA-like includes the inherent optimization of all on-device systems but excludes the new optimizations in §4.3.

Scenario-driven methodology. We conduct our evaluation by enacting the scenario from §2.1. An advertiser (Nike) runs ad

campaigns and repeatedly measures their efficacy. Each time a customer purchases quantity C of a product, Nike requests an attribution report, specifying the relevant ad campaigns. Nike requests reports over some attribution window and uses last-touch attribution. If no relevant impression is found, the report value is 0; otherwise, it is C . Nike batches reports and submits them to the aggregation service for a DP summation query using the Laplace mechanism. In our experiments, Nike repeatedly performs queries on report batches of size B , which varies by dataset. Once B reports are gathered, Nike runs its query. This is repeated over time as more batches of B reports are gathered. This is also repeated for each product, e.g., 10 in the microbenchmark/PATCG and a variable number in Criteo.

When requesting an attribution report for a conversion, Nike must specify the requested privacy budget, ϵ – the same value for all reports in a batch. Since the MPC uses the Laplace mechanism to ensure ϵ -DP, Nike selects ϵ to achieve acceptable accuracy. We assume Nike chooses ϵ in an attempt to keep query error within 5% ($\alpha = 0.05$) of the true value with 99% probability ($\beta = 0.01$), which corresponds to roughly 0.02 RMSRE. The formula for ϵ is: $\epsilon = \Delta \ln(1/\beta) / (\alpha \cdot B \cdot \tilde{c})$, where Δ is the maximum value for C and $\tilde{c}$ is Nike’s rough estimate of the average C .

Our specific method is: we run repeated, single-advertiser summation queries on fixed-size batches of attribution reports, using last-touch attribution and a privacy budget calibrated as described above. Default parameters include: a 7-day epoch size, a 30-day attribution window, and a global privacy budget per epoch of $\epsilon_G = 1$.

Microbenchmark dataset. To methodically evaluate Cookie Monster, under a range of conditions, more or less favorable to our optimizations, we create a synthetic dataset with 40,000 conversions across 10 products over 120 days. We expose two knobs: **Knob1**, the user participation rate per query, determines the fraction of users who are assigned conversions relevant for a particular query; **Knob2**, the number of impressions per user per day. These knobs impact budget allocation across IPA-like, ARA-like, and Cookie Monster. Lower Knob1 increases opportunities for fine-grained accounting in ARA-like and Cookie Monster. Lower Knob2 allows Cookie Monster to conserve privacy by not deducting from epochs with no relevant impressions, a key optimization over ARA-like.

PATCG dataset. To evaluate Cookie Monster under more realistic conditions, we resort to the PATCG and Criteo datasets. PATCG is a synthetic dataset released by the namesake W3C community group [31], which contains 24M conversions from a single advertiser over 30 days. This dataset represents a large advertiser, with only 1% of conversions attributed to impressions. There are 16M distinct users, and each user sees an average of 3.2 impressions. Users who convert take part in 1.5 conversions on average.

Criteo dataset. The Criteo dataset [41] is sampled from a 90-day log of live ad impressions and conversions recorded

by the Criteo ad-tech. The dataset includes data from 292 advertisers with 12M impression records and 1.3M conversion records. There are 10M unique users. The dataset provides opportunities for evaluating Cookie Monster in some additional dimensions compared to PATCG and the microbenchmark. In particular, the Criteo dataset contains data from multiple advertisers of widely distinct sizes, i.e., having a wide range in terms of number of impressions (1–2.6M impressions) and conversions per advertiser (0–478k conversions). However, since the dataset is heavily subsampled, missing many impressions, we also evaluate Cookie Monster on augmented versions of this dataset, in which we add synthetic impressions to compensate for the missing impressions that might otherwise favor Cookie Monster’s optimizations.

6.2 Microbenchmark Evaluation (Q1)

We use the microbenchmark to evaluate the impact of individual-sensitivity optimizations on privacy budget consumption across a range of controlled workloads (question Q1).

Varying user participation rate per query (knob1). We first vary the user participation rate per query. With a default batch size of 2,000 reports and 10 products (queried twice, totaling 20 queries), we create 40,000 conversions. Knob1 controls how these conversions are assigned to users, indirectly determining the total number of users. A lower knob1 favors on-device budgeting, as it spreads the 40,000 conversions across more users, creating more privacy filters for the advertiser. For example, with knob1 = 1, each user participates in all 20 query batches, requiring a minimum of 2,000 users, while knob1 = 0.001 generates 2M users. In the PATCG dataset, users convert with a 0.05 daily rate, corresponding to knob1 = 0.1, which we use as default in other experiments.

Fig. 4a and 4b show the average and maximum budget consumption across all device-epochs requested through the 20 queries. Qualitatively, the average budget consumption is a much more useful metric to assess the efficiency of the three systems, but we include the maximum because it reduces IDP guarantees to standard DP guarantees, thereby providing a more apples-to-apples comparison between on-device and off-device budgeting. Recall that IPA-like does not distribute budget consumption across devices but has a centralized privacy filter for each epoch, from which it deducts budget upon executing each query. As a result, increasing user participation per query (knob1) does not impact its budget consumption, which is always higher than the other methods’. Cookie Monster consistently consumes the least budget due to its optimizations, with greater improvements as user participation increases (lower knob1), since more device-epochs lack relevant impressions and don’t deduct budget. Even under the max budget metric, on-device systems outperform IPA-like, with Cookie Monster being the most efficient.

Varying the number of impressions per user per day (knob2). We now fix knob1 at 0.1 and vary the number of impressions per user per day (knob2). In PATCG, users see an

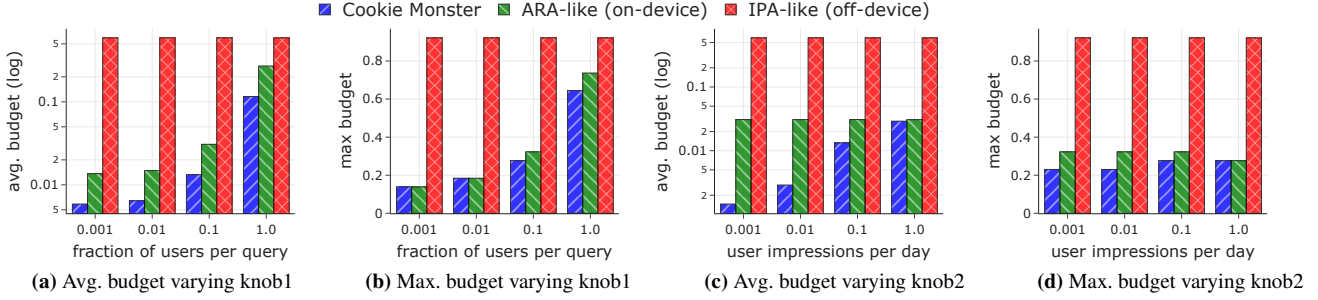


Fig. 4. Budget consumption on the microbenchmark. (a) and (b) show average and maximum budget consumption across all device-epochs, respectively, as a function of the fraction of users that participate per query (knob1); value of knob2 is constant 0.1. (c) and (d) show the same metrics as a function of user impressions per day (knob2); value of knob1 is constant 0.1.

average of 3.22 ads over 30 days, giving knob2 a value of 0.1. Fig. 4c and 4d confirm that Cookie Monster’s optimizations are most effective when users have fewer impressions.

Thus, Cookie Monster reduces budget consumption compared to baselines, especially when budget is spread across many users and when users have fewer impressions.

6.3 PATCG Evaluation (Q1, Q2)

We use the PATCG dataset to evaluate Cookie Monster’s impact on budget consumption (Q1) and query accuracy (Q2). This dataset links impressions and conversions to attributes, with values uniformly sampled from 0 to 9, representing 10 potential products. Nike queries each product eight times over the four months spanning the dataset, totaling 80 queries with batch sizes between 280,000 and 303,009 reports. Large batch sizes accommodate the low attribution rate (1% of impressions relevant to conversions), assuming Nike adjusts batch sizes accordingly.

Fig. 5a illustrates the average privacy budget consumed by each system as 80 queries are submitted for execution by the advertiser. The x-axis represents the order of queries, with points indicating budget consumption. IPA-like executes only a small fraction of queries (3.75%) due to its coarse-grained, population-level accounting, leading to early budget depletion. ARA-like and Cookie Monster, with finer-grained, individual-level accounting, execute all queries and resulting in smoother and lower average budget consumption. Cookie Monster shows up to 206 times lower average budget consumption compared to ARA-like, highlighting the benefits of its individual-sensitivity optimizations.

Next, we assess query accuracy (Q2). On-device systems (ARA-like and Cookie Monster) hide budgets when depleted, which can affect query accuracy, while IPA-like explicitly rejects queries with exhausted budgets. As in our experiments, privacy budgets are set to aim for high accuracy in the Laplace mechanism, we expect IPA’s executed queries to have errors within the 0.02 mark. In contrast, ARA and Cookie Monster may incur additional errors when epochs run out of budget, leading to nullified or incomplete reports.

Fig. 5b shows the CDF of root mean square relative error (RMSRE), defined as $\sqrt{\mathbb{E}[(M(D) - Q(D))^2 / Q(D)^2]}$ for

an estimate $M(D)$ of the query output $Q(D)$. This metric captures both Laplace-induced and IDP-bias-induced errors. The CDF shows query errors for each system. IPA-like’s line ends at 3.75% of queries, aligning with its budget constraints but maintaining within the 5% error mark. Cookie Monster consistently exhibits lower errors than ARA-like due to its budget conservation, resulting in fewer nullified reports and reduced bias. This is true without any bias mitigation strategies. In §6.5, we show that even with bias measurement running alongside every query, Cookie Monster still outperforms ARA-like (which has no bias measurement) in terms of budget consumption and query accuracy.

Finally, we explore how epoch length affects performance. Longer epochs strengthen device-epoch privacy guarantees but slow budget refreshing, leading to more query rejections in IPA and increased bias in on-device systems without mitigation. Fig. 5c evaluates RMSRE measures (median, first and third quartiles, and range) as epoch length varies. IPA-like’s query execution drops to 1.25% at one-month epochs, while Cookie Monster and ARA-like complete all queries but with increasing errors. Cookie Monster’s budget conservation results in fewer altered or nullified reports, maintaining lower error degradation compared to ARA-like as epochs grow.

6.4 Criteo Evaluation (Q1, Q2)

The Criteo dataset enables evaluation across diverse advertisers. It includes 1.3M conversions from 292 advertisers, with conversions ranging from 0 to 478k per advertiser. To achieve meaningful accuracy under DP, an advertiser needs a minimum number of reports. We set this minimum to 350, allowing us to formulate at least one query for 109 advertisers. Advertisers with more than 350 conversions wait to accumulate 350 reports per batch for each query, resulting in 898 queries across these advertisers using the attribute “product-category-3” as a product ID.

Fig. 6a shows a CDF of per-device average budget consumption across epochs, where the distribution covers all devices and all advertisers; that is, there is a single data point corresponding to each device and advertiser pair, which indicates the average consumption across epochs within an advertiser’s filters on a given device by the end of the workload. Lower

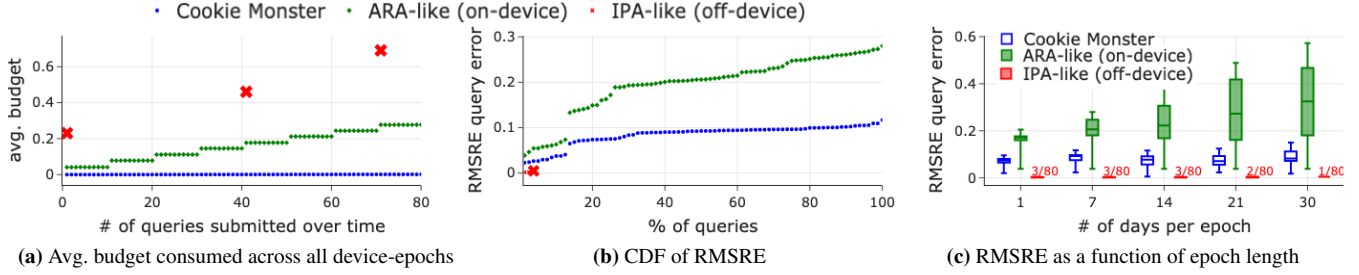


Fig. 5. Budget consumption and query accuracy on the PATCG dataset. (a) Average budget consumption across all device-epochs as a function of the number of queries submitted by the advertiser. (b) CDF of RMSRE with a 7-day epoch. (c) RMSRE median (horizontal lines), first and third quartiles (boxes), and max/min (top/bottom range markers) as epoch length increases.

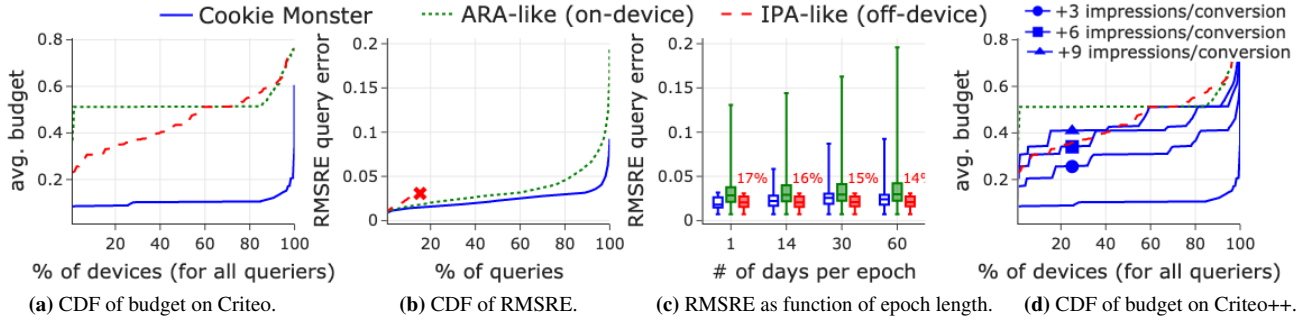


Fig. 6. Budget consumption and query accuracy on Criteo. (a) CDF of per-device average budget consumption across epochs for all devices and advertisers. (b) CDF of RMSREs for a 7-day epoch. (c) RMSRE metrics with varying epoch length (see Fig. 5c for format). (d) The same CDF as in (a), but for the Criteo++ dataset, showing the impact of synthetic impression augmentation on Cookie Monster's performance.

values indicate better performance. Cookie Monster conserves the most privacy budget, with 95% of device-advertiser pairs having more capacity left compared to both baselines.

Fig. 6b presents the CDF of RMSREs for all 898 queries. IPA-like completes only a small fraction of queries but with good accuracy. ARA-like and Cookie Monster accept all queries, potentially at the expense of higher error; however, Cookie Monster's error distribution remains better than ARA-like's, with errors within IPA-like's range for up to 96% of queries. This results from Cookie Monster's optimizations that conserve budget and avoid introducing bias.

Fig. 6c examines how RMSRE varies with epoch length. Longer epochs increase contention on per-epoch filters. Despite this, Cookie Monster's optimizations show substantial benefits, with minimal RMSRE increase (25% increase from 1-day to 60-day epoch for median RMSRE). Although maximum RMSRE increases with epoch length, Cookie Monster's performance remains superior to ARA-like.

Recall that the Criteo dataset is heavily subsampled, so there is the possibility that missing impressions may amplify the benefit of our optimizations. To assess Cookie Monster's performance in scenarios with more relevant impressions, we augment the Criteo dataset with synthetic impressions for each conversion. The results, shown in Fig. 6d, compare the CDFs of budget consumption with varying augmentation levels. The behavior of IPA-like and ARA-like remains unchanged by augmentation, as they do not optimize for missing relevant impressions. For Cookie Monster, budget efficiency

decreases as more synthetic impressions are added, approaching ARA-like's performance at 9 extra impressions per conversion. The impressions are uniformly distributed across the attribution window, ensuring that most epochs have relevant impressions for most conversions, so Cookie Monster's optimization is eliminated and its behavior follows ARA-like's.

6.5 Bias Measurement (Q3)

We evaluate Cookie Monster's bias measurement technique using our microbenchmark with default knob settings (0.1) and an increased query load to measure significant bias. Specifically, we use 60 days and repeat each query 40 times.

Fig. 7a shows the budget overhead incurred by bias measurement. The bias measurement's counts are scaled to have 10% the sensitivity of the original query, so the overall sensitivity of the query/side-query combination increases by 10%. The average consumed budget goes from 0.36 without bias measurement to 0.43 with bias measurement; this is more than a 10% increase since some epochs that originally paid zero budget through our IDP optimization, now pay for bias counts.

Fig. 7b shows the CDF of RMSREs across all 400 queries, with a log scale on the y-axis to highlight smaller differences among Cookie Monster variants compared to ARA. Due to the heavy query load, IPA executes only 5% of the queries and ARA ultimately returns empty reports, resulting in a relative error of 1. Cookie Monster without bias measurement plateaus at 0.2 error. Cookie Monster with bias measurement shows a similar trend to Cookie Monster without it, albeit

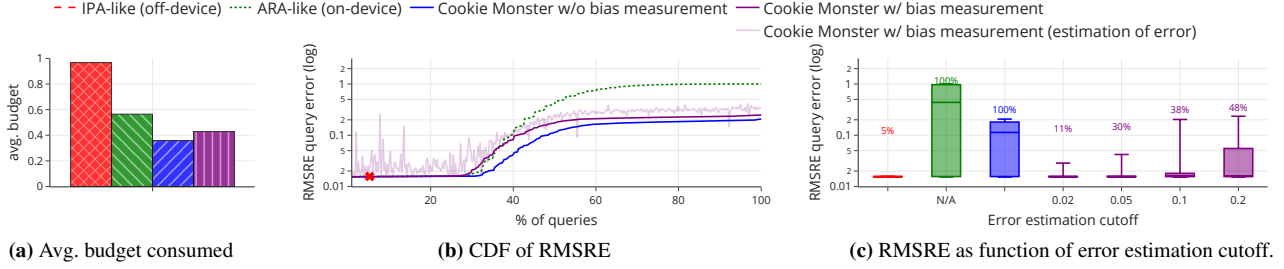


Fig. 7. Budget consumption and query accuracy with bias measurement on the microbenchmark. (a) Average budget consumed across all device-epochs. (b) CDF of true RMSRE for executed queries, alongside Cookie Monster’s RMSRE estimation from bias measurement (light-purple line). (c) Quartiles of true RMSRE, where queries with error estimate above a given cutoff are rejected by Cookie Monster with bias measurement.

with increased error, because the higher sensitivity of the query leads additional epochs to run out of budget. However, the bias measurements let queriers compute an estimate of the error, which, although noisy (as it is also differentially private), generally serves as an upper bound on true RMSRE. Queriers can compare this estimate to a predetermined cutoff and reject queries exceeding it. Fig. 7c displays the quartiles of true RMSREs after rejecting queries based on estimated RMSRE cutoffs. For instance, using a cutoff of 0.05 enables queriers to limit bias, achieving a maximum error of 0.04 (down from 0.21), but only accepting 30% of the queries. Rejected queries still consume budget, as rejection is a post-processing step.

Thus, even with rudimentary bias measurement, Cookie Monster offers substantial benefits over IPA while maintaining lower real error than ARA. While we validated our technique on a microbenchmark with increased query load, applying it to real-life datasets remains an open challenge. Future work could enhance our technique by scheduling bias measurements or using DP threshold comparison mechanisms.

7 Related Work

DP systems. Most DP systems operate in the centralized-DP model, where a trusted curator runs queries using global sensitivity [12]. Some implement fine-grained accounting through parallel composition [29, 26, 27, 25], a coarse form of individual DP (IDP) that lacks optimizations like those in Cookie Monster. Others function in the local-DP model, where devices randomize their data locally [23], and therefore inherently do on-device budgeting but have higher utility costs. Distributed systems like [38, 28] emulate the central model with cryptographic constructions; like IPA, they maintain a single privacy filter, not leveraging IDP to conserve budget. [4] uses the shuffle model [7] to combine local randomization with a minimal trusted party. Cookie Monster operates in the central model with on-device budgeting and uses an IDP formalization to enable new optimizations.

Private ads measurement. Several proposals exist for private ad measurement systems. Apple’s PCM [22] relies on entropy limits for privacy. Meta and Mozilla’s IPA [5] uses centralized budgeting, while Google’s ARA [3] and Apple’s

PAM [34] utilize on-device budgeting. ARA has primarily focused on optimizing in-query budget and utility. [10] optimizes a single vector-valued hierarchical query, whereas [1] assumes a simplified ARA with off-device impression-level DP guarantees, efficiently bounding each impression’s contribution for queries known upfront. [11] offers a framework for attribution logic and DP neighborhood relations, proposing clipping strategies for bounding global sensitivity. Our work optimizes on-device budgeting across queries, using tighter individual sensitivity bounds. Our method is agnostic to how these bounds are enforced, potentially benefiting from clipping algorithms [10, 1, 11].

IDP was introduced in the centralized-DP setting, where a trusted curator manages individual budgets and leverages individual sensitivity to optimize privacy accounting [13, 14]. IDP is used for SQL-like queries and gradient descent. The literature emphasizes the need to keep individual budgets private. [45] studies the release of DP aggregates over these budgets while [13] notes that out-of-budget records must be dropped silently, leaving bias analysis for future work.

8 Conclusion

Web advertising is at a crossroads, with a unique opportunity to enhance online privacy through new, privacy-preserving APIs from major browser vendors. We show that a novel individual DP formulation can significantly improve privacy budgeting in on-device systems. However, further progress is needed in query support, error management, and scalability. Our paper provides foundational insights and formal analysis to guide future research and industry collaboration.

Acknowledgements

We thank the anonymous reviewers for their constructive feedback and the Meta and Mozilla IPA teams, particularly Ben Savage and Martin Thompson, for their ongoing input. Special thanks to PATCG participants, especially Luke Winstrom, for their feedback on early proposals. This work was supported by NSF grants EEC-2133516, CNS-2106530, CNS-2104292, NSERC RGPIN-2022-04469, Google, Microsoft, Sloan Faculty Fellowships, and an Onassis Foundation Scholarship. Co-author Geambasu was partially employed by Meta during this project.

References

- [1] Hidayet Aksu et al. *Summary Reports Optimization in the Privacy Sandbox Attribution Reporting API*. Nov. 22, 2023. arXiv: 2311.13586 [cs].
- [2] Apple, Inc. *Apple announces powerful new privacy and security features*. <https://www.apple.com/newsroom/2023/06/apple-announces-powerful-new-privacy-and-security-features/>. 2023.
- [3] *Attribution Reporting API (ARA)*. <https://github.com/WICG/attribution-reporting-api/blob/main/AGGREGATE.md>. 2022.
- [4] Andrea Bittau et al. “Prochlo: Strong Privacy for Analytics in the Crowd”. In: *Proceedings of the 26th Symposium on Operating Systems Principles*. SOSP ’17. Shanghai, China: Association for Computing Machinery, 2017, pp. 441–459. ISBN: 9781450350853. DOI: 10.1145/3132747.3132769. URL: <https://doi.org/10.1145/3132747.3132769>.
- [5] Benjamin Case et al. *Interoperable Private Attribution: A Distributed Attribution and Aggregation Protocol*. Cryptology ePrint Archive, Paper 2023/437. <https://eprint.iacr.org/2023/437>. 2023. URL: <https://eprint.iacr.org/2023/437>.
- [6] Anthony Chavez. *A new path for Privacy Sandbox on the web*. <https://privacysandbox.com/news/privacy-sandbox-update/>. 2024.
- [7] Albert Cheu et al. “Distributed Differential Privacy via Shuffling”. In: *Advances in Cryptology – EUROCRYPT 2019*. Ed. by Yuval Ishai and Vincent Rijmen. Cham: Springer International Publishing, 2019, pp. 375–403. ISBN: 978-3-030-17653-2.
- [8] Google Chrome. *Federated Learning of Cohorts (FLoC)*. <https://privacysandbox.com/proposals/floc/>.
- [9] Google Chrome. *Protected Audience API overview*. <https://developers.google.com/privacy-sandbox/relevance/protected-audience>.
- [10] Matthew Dawson et al. *Optimizing Hierarchical Queries for the Attribution Reporting API*. Comment: Appeared at AdKDD 2023 workshop; Final proceedings version. Nov. 27, 2023. arXiv: 2308.13510 [cs].
- [11] John Delaney et al. *Differentially Private Ad Conversion Measurement*. 2024. arXiv: 2403.15224 [cs, CR].
- [12] Cynthia Dwork and Aaron Roth. “The Algorithmic Foundations of Differential Privacy”. In: *Foundations and Trends® in Theoretical Computer Science* 9.3–4 (2013), pp. 211–407. ISSN: 1551-305X, 1551-3068. DOI: 10.1561/04000000042.
- [13] Hamid Ebadi, David Sands, and Gerardo Schneider. “Differential Privacy: Now It’s Getting Personal”. In: *Proceedings of the 42nd Annual ACM SIGPLAN - SIGACT Symposium on Principles of Programming Languages*. POPL ’15: The 42nd Annual ACM SIGPLAN SIGACT Symposium on Principles of Programming Languages. Mumbai India: ACM, Jan. 14, 2015, pp. 69–81. ISBN: 978-1-4503-3300-9. DOI: 10.1145/2676726.2677005.
- [14] Vitaly Feldman and Tijana Zrnic. “Individual Privacy Accounting via a Rényi Filter”. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 28080–28091.
- [15] Badih Ghazi et al. “Deep Learning with Label Differential Privacy”. In: *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., 2021, pp. 27131–27145.
- [16] Google. *Attribution Reporting API with Aggregatable Reports*. <https://github.com/WICG/attribution-reporting-api/blob/main/AGGREGATE.md#contribution-bounding-and-budgeting/>. 2024.
- [17] *Google’s FLoC Is a Terrible Idea*. <https://www.eff.org/deeplinks/2021/03/googles-floc-terrible-idea>. 2021.
- [18] *Hybrid Proposal*. <https://github.com/patcg-individual-drafts/hybrid-proposal>. 2024.
- [19] *iCloud Private Relay Overview*. https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf. 2021.
- [20] *Intelligent Tracking Prevention 2.3*. <https://webkit.org/blog/9521/intelligent-tracking-prevention-2-3/>. 2019.
- [21] *Interoperable Private Attribution (IPA)*. <https://github.com/patcg-individual-drafts/ipa>. 2022.
- [22] *Introducing Private Click Measurement, PCM*. <https://webkit.org/blog/11529/introducing-private-click-measurement-pcm/>. 2021.
- [23] Shiva Prasad Kasiviswanathan et al. “What Can We Learn Privately?” In: *SIAM Journal on Computing* 40.3 (2011), pp. 793–826. DOI: 10.1137/090756090. URL: <https://doi.org/10.1137/090756090>.
- [24] Daniel Kifer et al. *Guidelines for Implementing and Auditing Differentially Private Systems*. Tech. rep. 2020.
- [25] Nicolas Küchler et al. “CoHERE: Privacy Management in Large Scale Systems”. In: *CoRR* abs/2301.08517 (2023). DOI: 10.48550/ARXIV.2301.08517. arXiv: 2301.08517. URL: <https://doi.org/10.48550/arXiv.2301.08517>.
- [26] Mathias Lecuyer et al. “Privacy Accounting and Quality Control in the Sage Differentially Private Machine Learning Platform”. In: *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. 2019.
- [27] Tao Luo et al. “Privacy Budget Scheduling”. In: *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, July 2021, pp. 55–74. ISBN: 978-1-939133-22-9.

- URL: <https://www.usenix.org/conference/osdi21/presentation/luo>.
- [28] Elizabeth Margolin et al. “Arboretum: A Planner for Large-Scale Federated Analytics with Differential Privacy”. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. SOSP ’23. , Koblenz, Germany, Association for Computing Machinery, 2023, pp. 451–465. ISBN: 9798400702297. DOI: 10.1145/3600006.3624566. URL: <https://doi.org/10.1145/3600006.3624566>.
 - [29] Frank D. McSherry. “Privacy Integrated Queries: An Extensible Platform for Privacy-Preserving Data Analysis”. In: *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’09. New York, NY, USA: Association for Computing Machinery, June 29, 2009, pp. 19–30. ISBN: 978-1-60558-551-2. DOI: 10.1145/1559845.1559850.
 - [30] *Over a decade of anti-tracking work at Mozilla*. <https://blog.mozilla.org/en/privacy-security/mozilla-anti-tracking-milestones-timeline/>. 2022.
 - [31] *PATCG Attribution Synthetic Data*. https://docs.google.com/document/d/1Vxq4LrMe3A2Wllu-7IYP1Hycr_nz3_qTpAICX9fLcw. 2024.
 - [32] *Privacy Preserving Ad Click Attribution For the Web*. <https://webkit.org/blog/8943/privacy-preserving-ad-click-attribution-for-the-web/>. 2019.
 - [33] *Privacy-Preserving Attribution: Level 1*. <https://private-attribution.github.io/api/>. 2024.
 - [34] *Private Ad Measurement (PAM)*. <https://github.com/patcg-individual-drafts/private-ad-measurement>. 2023.
 - [35] *Private Advertising Technology Community Group*. <https://www.w3.org/community/patcg>. 2024.
 - [36] Ryan Rogers et al. “Privacy odometers and filters: pay-as-you-go composition”. In: *Proceedings of the 30th International Conference on Neural Information Processing Systems*. NIPS’16. Barcelona, Spain: Curran Associates Inc., 2016, pp. 1929–1937. ISBN: 9781510838819.
 - [37] Ryan M Rogers et al. “Privacy Odometers and Filters: Pay-as-you-go Composition”. In: *Advances in Neural Information Processing Systems*. Ed. by D. Lee et al. Vol. 29. Curran Associates, Inc., 2016.
 - [38] Edo Roth et al. “Orchard: differentially private analytics at scale”. In: *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. OSDI’20. USA: USENIX Association, 2020. ISBN: 978-1-939133-19-9.
 - [39] Google Privacy Sandbox. *Privacy Sandbox for the Web*. https://privacysandbox.com/intl/en_us/open-web. 2023.
 - [40] *Selenium*. <https://www.selenium.dev/>. 2024.
 - [41] Marcelo Tallis and Pranjul Yadav. “Reacting to Variations in Product Demand: An Application for Conversion Rate (CR) Prediction in Sponsored Search”. In: *arXiv preprint arXiv:1806.08211* (2018).
 - [42] *Understanding Apple’s Private Click Measurement*. <https://blog.mozilla.org/en/mozilla/understanding-apples-private-click-measurement/>. 2022.
 - [43] Salil Vadhan and Wanrong Zhang. “Concurrent Composition Theorems for Differential Privacy”. In: *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*. STOC 2023. Orlando, FL, USA: Association for Computing Machinery, 2023, pp. 507–519. ISBN: 9781450399135. DOI: 10.1145/3564246.3585241. URL: <https://doi.org/10.1145/3564246.3585241>.
 - [44] *Weighted Aggregate Logistic Regression*. https://github.com/patcg-individual-drafts/ipa/blob/main/logistic_regression.md. 2024.
 - [45] Da Yu et al. “Individual Privacy Accounting for Differentially Private Stochastic Gradient Descent”. In: *Transactions on Machine Learning Research* (Apr. 27, 2023). ISSN: 2835-8856.